

Credit Scoring sobre el dataset German Credit Data

Clasificación supervisada con Random Forest y consulta documental con RAG sobre normativa de riesgo crediticio

Alejandro Tirado Ramirez Jeronimo Restrepo Giraldo
atirador1@eafit.edu.co jrestrepg1@eafit.edu.co

Escuela de Ciencias Aplicadas e Ingeniería . Universidad EAFIT
Curso: Inteligencia Artificial . Semestre 2026-1
Entrega: 19–22 de mayo de 2026

Repositorio: <https://github.com/Alejoso/ProyectoFinal-IA>

Resumen

Contexto: El credit scoring es una técnica estadística fundamental para la gestión del riesgo crediticio, regulada en Colombia por el SARC (Superintendencia Financiera) y a nivel internacional por los acuerdos de Basilea. **Objetivo:** Construir un clasificador supervisado capaz de identificar solicitantes con alta probabilidad de incumplimiento sobre el dataset German Credit Data, complementado con un sistema RAG que consulte documentación regulatoria del dominio. **Método:** EDA y preprocesamiento (encoding ordinal/one-hot diferenciado, escalado, división estratificada), comparación de tres modelos clásicos con validación cruzada estratificada de 5 folds, tuning con GridSearchCV sobre Random Forest, y un sistema RAG con sentence-transformers multilingüe, ChromaDB y llama-3.3-70b-versatile vía Groq sobre un corpus mixto (12 documentos curados + Basilea II + SARC Colombia). **Resultados:** El modelo final (Random Forest tuneado) alcanzó **ROC-AUC = 0,8087** y **F1 = 0,5983** en test, superando significativamente al baseline trivial. **Conclusión:** El sistema integrado articula un clasificador interpretable con una capa de consulta documental que aporta contexto regulatorio en lenguaje natural, manteniendo trazabilidad de las fuentes.

1. Introducción

La evaluación automatizada del riesgo de crédito es uno de los problemas mejor estudiados de la aplicación práctica del *machine learning* en finanzas. A pesar de la disponibilidad de algoritmos cada vez más sofisticados, las decisiones de otorgamiento deben seguir siendo **interpretables** y **auditables** por imposición regulatoria, tanto bajo el marco de Basilea II como bajo la Circular Básica Contable y Financiera de la Superintendencia Financiera de Colombia.

Este proyecto aborda dos preguntas complementarias:

“Puede un modelo de clasificación supervisada predecir la condición de mal pagador sobre el dataset German Credit Data con un ROC-AUC superior a 0,75, considerando el desbalance de clases del 70/30?”

Y de forma complementaria:

“Es viable integrar un sistema RAG que consulte documentación regulatoria del dominio para apoyar la interpretación del modelo y de sus resultados?”

El dataset utilizado es *German Credit Data* del UCI Machine Learning Repository: 1.000 registros de solicitantes en Alemania, con 20 variables predictoras (13 categóricas, 7 numéricas) y una variable objetivo binaria. El documento se organiza así: la Sección 2 describe el EDA, la 3 la arquitectura, la 4 la metodología, la 5 los resultados, la 6 la discusión y la 7 las conclusiones.

2. Datos y Exploración (EDA)

2.1. Descripción del dataset

Cuadro 1: Resumen del dataset utilizado

Atributo	Descripción
Fuente	https://archive.ics.uci.edu/dataset/144/
Registros (N)	1.000 filas
Features (p)	7 numéricas / 13 categóricas
Variable objetivo	target (binaria: 0=bueno, 1=mallo)
Período / Versión	Statlog (German Credit Data)
Tamaño	~30 KB

Se utilizó el archivo `german.data` (códigos categóricos crípticos como A11, A34) en lugar de `german.data-numeric` (versión preprocesada por los

autores). Esta decisión permite tomar y justificar las decisiones de encoding como parte del proyecto, en lugar de heredarlas opacamente.

2.2. Hallazgos del análisis exploratorio

- **Desbalance moderado.** 70/30 (700 buenos vs 300 malos). Implicación: *accuracy* es engañosa, un modelo trivial alcanza 70 % sin aprender nada.
- **Asimetría en numéricas.** *monto_credito* (media 3.271, mediana 2.319, máx 18.424) y *duracion_meses* (4-72) están sesgadas a la derecha.
- **Variables más discriminativas.** *estado_cuenta*, *historial_credito*, *proposito* y *ahorros* con χ^2 altos y *p*-valores $< 0,001$.
- **Hallazgo contraintuitivo.** *historial_credito = sin_creditos* tiene tasa de incumplimiento *mayor* que *cuenta_critica_otros_bancos*. Patrón no monótono que justifica one-hot encoding.
- **Multicolinealidad leve.** Correlación de $\sim 0,62$ entre *duracion_meses* y *monto_credito*.
- **Sin valores nulos ni duplicados.**



Figura 1: Distribución del target (70/30)

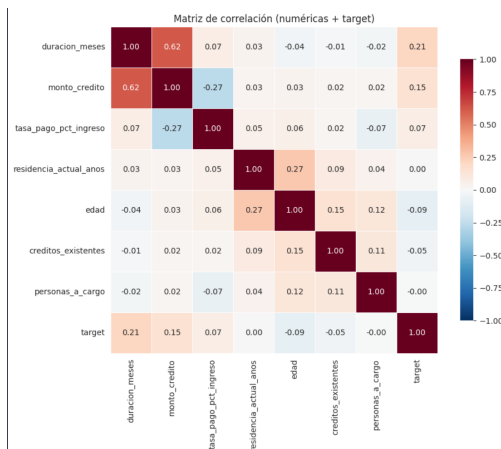


Figura 2: Correlaciones de las variables numericas

3. Arquitectura del Sistema

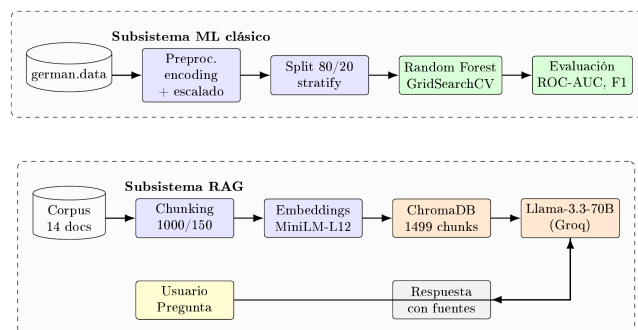


Figura 3: Arquitectura general: subsistema de ML clásico (arriba) y subsistema RAG (abajo)

El sistema integra dos subsistemas complementarios: un clasificador supervisado entrenado sobre el dataset tabular, y un sistema RAG que responde preguntas en lenguaje natural sobre el dominio. Comparten vocabulario (las variables del dataset son discutidas en el corpus del RAG) pero operan de manera independiente.

3.1. Componentes principales

Preprocesamiento. Pipeline en sklearn con ColumnTransformer: StandardScaler para 7 numéricas, OrdinalEncoder con categorías ordenadas para 4 variables ordinales (*estado_cuenta*, *ahorros*, *empleo_actual*, *trabajo*), y OneHotEncoder con *drop='first'* para 9 nominales. División estratificada 80/20 (*stratify=y*, *random_state=42*). El preprocesador se ajusta solo sobre train para evitar *data leakage*. Resultado: 38 features tras encoding.

Modelo de ML. Random Forest con *class_weight='balanced'* para tratar el desbalance, tuneado vía GridSearchCV sobre 108 combinaciones (*n_estimators* $\in \{100, 200, 300\}$, *max_depth* $\in \{5, 10, 15, \text{None}\}$, *min_samples_split* $\in \{2, 5, 10\}$, *min_samples_leaf* $\in \{1, 2, 4\}$). Elegido por su capacidad de capturar relaciones no lineales e interpretabilidad vía *feature importance*.

Componente LLM. llama-3.3-70b-versatile vía API de Groq (free tier), con *temperature=0,1* para favorecer fidelidad al contexto recuperado por encima de creatividad. Estrategia: *zero-shot* con instrucciones explícitas de citación de fuentes y rechazo de información ausente del contexto.

RAG. Vector store: ChromaDB con persistencia local. Embeddings: paraphrase-multilingual-MiniLM-L12-v2 de sentence-transformers (384 dim, multilingüe), elegido por compatibilidad con corpus mixto español-inglés.

Chunking: RecursiveCharacterTextSplitter con chunk_size=1000 y chunk_overlap=150. Recuperación de $k = 8$ chunks por consulta. Corpus indexado en 1.499 chunks totales.

4. Metodología

4.1. Configuración experimental

Cuadro 2: Hiperparámetros del modelo final (Random Forest tuneado)

Parámetro	Valor
Algoritmo	RandomForestClassifier (sklearn)
n_estimators	100 (seleccionado por GridSearch)
max_depth	15
min_samples_split	10
min_samples_leaf	1
class_weight	'balanced'
random_state	42
Criterio de selección	ROC-AUC en CV 5-fold estratificado
Framework	scikit-learn 1.5.2
Hardware	CPU local (WSL, Ryzen 5 9600X)

4.2. Estrategia de validación

Se utilizó **Stratified k -fold cross-validation** con $k = 5$, apropiada para problemas binarios desbalanceados: la estratificación garantiza que cada fold mantenga la proporción 70/30. El conjunto de prueba (20 %, $n = 200$) se mantuvo intocado hasta la evaluación final, evitando sobreajuste indirecto.

La elección de **ROC-AUC** como métrica principal responde al estándar de la industria en credit scoring: es independiente del umbral de decisión y mide la capacidad de *ranking* del modelo. Como secundarias: *precision*, *recall* y F1 de la clase positiva, priorizando recall por el mayor costo de los falsos negativos.

4.3. Experimentos realizados

1. **Baseline (DummyClassifier)**. Predice siempre clase mayoritaria.
2. **Logistic Regression** con `class_weight='balanced'`.
3. **Random Forest** con configuración por defecto (`n_estimators=200, max_depth=10`).
4. **Gradient Boosting** con configuración estándar.
5. **Random Forest tuneado** (GridSearchCV: 108 combinaciones $\times$ 5 folds = 540 ajustes).

5. Resultados

5.1. Métricas de evaluación

Cuadro 3: Comparación de modelos – CV estratificada 5-fold (train)

Modelo	AUC	F1	Recall
Logistic Regression	0,7741	0,5843	0,6833
Random Forest	0,7933	0,5418	0,4750
Gradient Boosting	0,7881	0,5077	0,4417

Tras el GridSearch, los mejores hiperparámetros fueron `n_estimators=100, max_depth=15, min_samples_split=10, min_samples_leaf=1`. La Tabla 4 reporta las métricas finales sobre test ($n = 200$).

Cuadro 4: Métricas del modelo final (RF tuneado) en test set

Métrica	Valor
Accuracy	0,7650
Precision	0,6140
Recall	0,5833
F1-score	0,5983
ROC-AUC	0,8087

5.2. Matriz de confusión y curva ROC

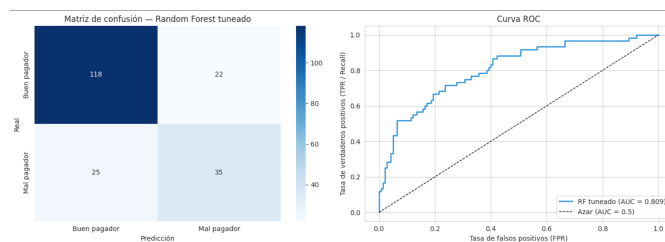


Figura 4: Matriz de confusión (izquierda) y curva ROC (derecha) del Random Forest tuneado. AUC=0,809.

La matriz arroja: TN=118, FP=22, FN=25, TP=35. Interpretación de negocio:

- De 60 malos pagadores reales, el modelo identificó 35 (recall 58,3 %) y dejó pasar 25.
- De 140 buenos pagadores, rechazó injustamente a 22 (15,7 % FPR).

5.3. Importancia de variables

Cuadro 5: Top 10 features por importancia (RF tuneado)

Feature	Importancia
estado_cuenta	0,1526
monto_credito	0,1181
duracion_meses	0,0875
edad	0,0814
ahorros	0,0587
empleo_actual	0,0517
tasa_pago_pct_ingreso	0,0354
historial_critico_otros	0,0344
otros_planes_pago_ninguno	0,0297
residencia_actual_anos	0,0283

estado_cuenta resultó la más predictiva (15,3% de la importancia), seguida por monto_credito y duracion_meses. Esta lista coincide con las variables identificadas por el EDA, dando consistencia entre exploración y modelo.

5.4. Análisis cualitativo del RAG

El sistema responde con citación explícita de fuentes:

```

1 # Pregunta del usuario
2 query = "Que son PD, LGD y EAD segun
3 el enfoque IRB de Basilea II?"
4
5 # Respuesta del sistema (resumida)
6 {
7   "respuesta": "PD es la probabilidad de
8   incumplimiento del deudor en un año.
9   LGD es la pérdida en caso de default,
10  como % del monto expuesto. EAD es el
11  monto total expuesto al default. La
12  pérdida esperada: EL = PD x LGD x EAD.",
13  "fuentes": [
14    "03_pd_lgd_ead_basilea.md",
15    "bcbs128.pdf"
16  ],
17  "chunks_usados": 8
18 }
```

Listing 1: Ejemplo de output del sistema RAG

Ante preguntas fuera del corpus ("¿cuál es el precio del dólar hoy?"), el sistema responde explícitamente que no tiene información suficiente, sin alucinar. Esto se logró con instrucciones explícitas.

6. Discusión

6.1. Interpretación de resultados

El objetivo se cumplió: ROC-AUC = 0,8087 en test, superior al umbral 0,75 y dentro del rango típico para German Credit (0,75-0,82). El AUC en test resultó incluso superior al de CV (0,7933), indicando que el tuning fue efectivo sin sobreajuste a los folds.

Tradeoff observado entre métricas. Comparado con Logistic Regression sin tunear (recall CV = 0,6833), el RF tuneado tiene mejor AUC pero menor recall (0,5833). Refleja una decisión deliberada del GridSearch: se optimizó por AUC y se sacrificó recall. En producción bancaria, donde el costo de un falso negativo se estima entre 3 y 5 veces el de un falso positivo, sería razonable bajar el umbral por debajo de 0,5 o re-tunear optimizando por F1.

Coherencia EDA-modelo. Las variables más importantes (estado_cuenta, monto_credito, duracion_meses, historial_credito, ahorros) son las que el EDA identificó vía χ^2 y tasas de incumplimiento. Señal positiva: el modelo aprende patrones ya visibles estadísticamente.

6.2. Limitaciones

- **Tamaño.** 1.000 registros es pequeño para estándares modernos, aumenta varianza.
- **Antigüedad.** Dataset alemán de los 90, no representativo del contexto colombiano actual; uso metodológico.
- **RAG sin evaluación cuantitativa.** Validación cualitativa solamente.
- **Corpus RAG desbalanceado.** Basilea II (350 págs) domina sobre corpus generado y SARC; mitigado con $k = 8$.

6.3. Trabajo futuro

1. **Optimización por costo asimétrico.** Reentrenar minimizando pérdida que pondere FN con costo 3-5x mayor que FP.
2. **Evaluación cuantitativa del RAG.** Introducir RAGAS (faithfulness, answer relevancy) sobre preguntas anotadas.
3. **Interpretabilidad individual.** SHAP sobre el RF para explicar predicciones de casos, con el RAG traduciendo al lenguaje del SARC.

7. Conclusiones

El proyecto integró un clasificador supervisado y un sistema RAG sobre el dominio del credit scoring. El modelo final alcanzó ROC-AUC = 0,8087 en test sobre German Credit Data, superando claramente al baseline trivial (AUC = 0,5) y respondiendo afirmativamente a la pregunta de investigación.

El sistema RAG, sobre corpus mixto de documentación curada y normativa oficial (Basilea II + SARC Colombia), responde consultas en lenguaje natural con citación de fuentes y trazabilidad. La integración entre subsistemas no es directa pero comparten vocabulario, abriendo el camino a la explicabilidad asistida del trabajo futuro.

La consistencia entre EDA y la importancia de variables del modelo, junto con el comportamiento esperado del RAG frente a preguntas fuera de su corpus, sugiere que el pipeline metodológico es sólido y que las decisiones de diseño (encoding diferenciado, validación estratificada, tratamiento del desbalance, prompt estricto del LLM) fueron acertadas.

Contribuciones del equipo

Integrante	Contribución principal
Alejandro Tirado	EDA, preprocesamiento, visualizaciones, Modelado ML
Jerónimo Restrepo	Tuning, evaluación y visualización en Streamlit

Distribución de responsabilidades por integrante.

Repositorio y Demo

- **GitHub:** <https://github.com/Alejoso/ProyectoFinal-IA>
- **Video demo (≤3 min):** [https://youtu.be/\[ID\]](https://youtu.be/[ID])
- **Instalación:** `pip install -r requirements.txt`
- **Notebooks:** 01_eda.ipynb, 02_preprocessing.ipynb, 03_ml_classifiers.ipynb, 04_rag.ipynb

Referencias

- [1] Basel Committee on Banking Supervision. (2006). *International Convergence of Capital Measurement and Capital Standards: A Revised Framework – Comprehensive Version*. Bank for International Settlements. <https://www.bis.org/publ/bcbs128.htm>
- [2] Superintendencia de la Economía Solidaria. (2023). *Circular Básica Contable y Financiera, Capítulo II: Sistema de Administración del Riesgo de Crédito – SARC*. https://www.supersolidaria.gov.co/sites/default/files/data/20230817_capitulo_ii.pdf
- [3] Hofmann, H. (1994). *Statlog (German Credit Data)*. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data>
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
- [5] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [6] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. <https://arxiv.org/abs/1908.10084>
- [7] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR, 12, 2825–2830.